

SupportIQ — Technical Architecture Reference

Audience: AI Architects, Platform Engineers, Technical Decision Makers

1. System Overview

SupportIQ is a **production-architecture AI platform** built on top of the TaskFlow Spring Boot API. It demonstrates a real-world pattern for integrating a large language model into an existing enterprise Java application — without introducing a Python layer, without a vector database, and without any vendor-specific AI SDK.

The core thesis: **AI should be a callable component, not a rewrite.** The entire AI integration is a thin synchronous gateway over a standard HTTP REST client. No streaming. No embeddings. No orchestration framework. Just structured prompts, typed JSON responses, and a clean error boundary.

2. Technology Stack

Layer	Technology	Version
Runtime	Java	17
Framework	Spring Boot	4.1.0
Web layer	Spring WebMVC (Servlet)	7.x
Persistence	Spring Data JPA + Hibernate	7.x
Database	H2 (in-memory)	Runtime
HTTP client (AI calls)	Spring RestClient	Synchronous, blocking
JSON parsing	Jackson ObjectMapper	2.x
Validation	Jakarta Validation (Bean Validation 3.0)	—
API docs	SpringDoc OpenAPI / Swagger UI	2.8.9
Build	Maven	—
Boilerplate reduction	Lombok (@Slf4j, @Builder, @Data)	—
AI proxy	EPAM DIAL (ai-proxy.lab.epam.com)	OpenAI-compatible
AI model	gpt-5-mini-2025-08-07	via DIAL
Frontend	Vanilla JS SPA (single HTML file)	Served by Spring Boot static

Key dependency constraint: No LangChain, no Spring AI, no Anthropic SDK. The AI integration uses only `spring-boot-starter-webmvc` (which includes `RestClient`). This is intentional — it proves the pattern is achievable with zero AI-specific dependencies.

3. Repository & Package Structure

```
com.epam.taskflow.taskflow_api
├─ config/
│   ├── AiConfig.java           - RestClient bean wired to DIAL endpoint + API key
│   ├── OpenApiConfig.java      - Swagger/OpenAPI configuration
│   └─ SupportDataInitializer.java - Seed 20 demo tickets on startup (@Profile("!test"))
└─
```

```

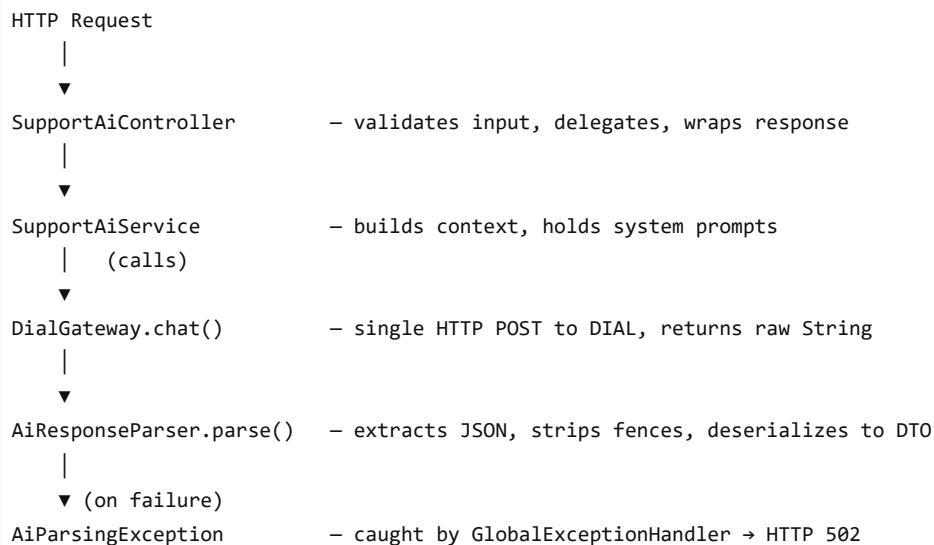
├─ controller/
│   ├─ TaskController.java      - /api/tasks/** CRUD
│   ├─ NoteController.java     - /api/tasks/{id}/notes CRUD
│   ├─ AiController.java       - /api/ai/ask (RAG Q&A)
│   ├─ SupportTicketController.java - /api/support/tickets CRUD + filters
│   └─ SupportAiController.java - /api/support/ai/* and triage endpoints
│
├─ service/
│   ├─ TaskService.java
│   ├─ NoteService.java
│   ├─ AiService.java           - RAG: reads .java files, passes as context to DIAL
│   ├─ SupportTicketService.java - Ticket CRUD + business logic
│   ├─ SupportAiService.java    - ALL 5 AI methods, all 4 system prompts
│   ├─ DialGateway.java         - Shared @Component: wraps RestClient → DIAL HTTP call
│   └─ AiResponseParser.java    - JSON extraction + type-safe deserialization
│
├─ exception/
│   ├─ GlobalExceptionHandler.java - Single @RestControllerAdvice for all errors
│   ├─ ResourceNotFoundException.java → 404
│   ├─ AiParsingException.java     → 502 (AI returned unparseable output)
│   └─ ErrorResponse.java         - Uniform error DTO with timestamp, status, path
│
├─ model/
│   ├─ Task.java                - JPA entity (tasks table)
│   ├─ Note.java                - JPA entity (notes table)
│   └─ SupportTicket.java       - JPA entity (support_tickets table)
│
├─ dto/                         - Request/response DTOs (entities never leave service layer)
├─ mapper/                      - Entity ↔ DTO mapping (hand-written, no MapStruct)
└─ repository/                  - Spring Data JPA interfaces

```

4. AI Integration Architecture

4.1 Design Principle: The Thin AI Layer

The AI integration follows a **single-responsibility chain**:



Each component has exactly one job. `SupportAiService` never touches HTTP. `DialGateway` never knows about prompt design. `AiResponseParser` never knows about domain objects.

4.2 DialGateway — The AI Boundary Component

```
@Component
public class DialGateway {
    public String chat(String systemPrompt, String userMessage)
}
```

What it does:

- Takes a system prompt and user message
- Constructs the OpenAI `chat/completions` request body as a `Map<String, Object>`
- POSTs to `{DIAL_ENDPOINT}/openai/deployments/{model}/chat/completions?api-version={version}`
- Extracts and returns `choices[0].message.content` as a raw `String`

Wire format sent to DIAL:

```
{
  "messages": [
    { "role": "system", "content": "<system prompt>" },
    { "role": "user", "content": "<user message / context>" }
  ],
  "max_completion_tokens": 4096
}
```

Key design decisions:

- `RestClient` (Spring 6.1+) not `RestTemplate` — cleaner builder API, built into WebMVC starter
- No `temperature`, no `top_p` — model defaults used deliberately (consistent, reproducible outputs)
- `max_completion_tokens: 4096` — large enough for bulk triage payloads (20 tickets × 200 chars each)
- `@Component` not `@Service` — semantically a gateway/adaptor, not a business service
- `ParameterizedTypeReference<Map<String, Object>>` for response — avoids a vendor-specific DTO dependency

DIAL configuration (`application.properties`):

```
dial.endpoint=https://ai-proxy.lab.epam.com
dial.api-key=dial-ionv90qrjw2pdyw9evpeml1iwzh
dial.model=gpt-5-mini-2025-08-07
dial.api-version=2025-04-01-preview
```

`AiConfig.java` wires this:

```
@Bean
public RestClient dialRestClient(
    @Value("${dial.endpoint}") String dialEndpoint,
    @Value("${dial.api-key}") String dialApiKey) {
    return RestClient.builder()
        .baseUrl(dialEndpoint)
        .defaultHeader("Api-Key", dialApiKey)
        .build();
}
```

The `Api-Key` header is DIAL's auth mechanism. Standard `Authorization: Bearer` is not used here — this is EPAM's internal gateway convention.

4.3 AiResponseParser — Robust JSON Extraction

LLMs do not reliably return bare JSON even when instructed to. The parser handles three real output shapes:

```
Shape 1 – Bare JSON:      {"category":"BILLING",...}
Shape 2 – Fenced JSON:    ```json\n{"category":"BILLING",...}\n```
Shape 3 – Prose + JSON:   "Here is the result: {...}" (first { to last )
```

Implementation:

```
private String extractJson(String raw) {
    // 1. Try markdown fence pattern: ```json\n...\n```
    Matcher matcher = FENCE_PATTERN.matcher(raw);
    if (matcher.find()) return matcher.group(1).trim();

    // 2. Find first { or [ and last matching } or ]
    int start = findFirst(raw, '{', '[');
    char close = open == '{' ? '}' : ']';
    int end = raw.lastIndexOf(close);
    return raw.substring(start, end + 1).trim();
}
```

After extraction, Jackson `ObjectMapper.readValue()` deserializes into the target DTO. If either step fails, `AiParsingException` is thrown — propagating to `GlobalExceptionHandler` → HTTP 502.

Two parse overloads:

```
public <T> T parse(String raw, Class<T> targetType) // direct class
public <T> T parse(String raw, TypeReference<T> typeRef) // generic types (e.g. List<T>)
```

Note on ObjectMapper instantiation: The `ObjectMapper` in `AiResponseParser` is a direct field: `private final ObjectMapper objectMapper = new ObjectMapper()`. It is **not Spring-injected**. This avoids potential circular bean definition issues and is safe because `ObjectMapper` is thread-safe after construction with default settings.

4.4 Prompt Engineering Architecture

All system prompts are `private static final String` constants in `SupportAiService`. This is deliberate — prompts are code, not configuration. They live alongside the logic that uses them, version-controlled, reviewable, testable.

Prompt structure (all 4 follow the same pattern):

- | | |
|----------------------|--|
| 1. ROLE DEFINITION | – "You are an expert customer support intelligence AI..." |
| 2. ENUM CONSTRAINTS | – Explicitly list valid enum values the model must use |
| 3. SCORING RULES | – Define numeric scales with labeled breakpoints |
| 4. ESCALATION LOGIC | – Explicit boolean trigger conditions |
| 5. SCHEMA CONTRACT | – "Respond ONLY with valid JSON – no markdown, no explanation" |
| 6. EXACT JSON SCHEMA | – Inline schema string as the final line of the prompt |

Example — Analysis prompt schema contract (actual production code):

```
"Respond ONLY with valid JSON – no markdown, no explanation – matching exactly this schema:
{"category\:string,\"subcategory\:string,\"sentimentScore\:int,\"sentimentLabel\:string,
\"riskScore\:int,\"escalationRequired\:boolean,\"escalationReason\:string|null,
\"suggestedPriority\:string,\"keyIssues\":[string],\"recommendedAction\:string,
\"estimatedResolutionTime\:string,\"sentimentAnalysis\:string,\"urgencyFactors\":[string]}"
```

Why inline schema strings instead of generating them from Java DTOs?

Two reasons:

1. DTOs contain Jackson annotations and validation annotations that pollute schema generation
2. The schema string is part of the model instruction — it must be human-readable at the prompt level for prompt debugging. Generating it automatically obscures that.

Sentiment scoring constraint example:

```
SENTIMENT SCORE: 1 (VERY_ANGRY) to 10 (VERY_SATISFIED).  
Use: 1-2=VERY_ANGRY, 3-4=ANGRY, 5-6=NEUTRAL, 7-8=SATISFIED, 9-10=VERY_SATISFIED
```

Providing the full label-to-range mapping eliminates model ambiguity and makes outputs deterministic enough for consistent UI color coding.

4.5 AI Data Flow — Ticket Analysis (Full Trace)

```
POST /api/support/tickets/{id}/analyze  
|  
▼  
SupportAiController.analyzeTicket(id)  
| @Valid not needed – path variable, no bean to validate  
▼  
SupportAiService.analyzeTicket(Long ticketId)  
|  
| ticketService.findOrThrow(id) → SupportTicket entity  
|  
| buildTicketContext(ticket) → structured plain-text string:  
|   "Ticket: TKT-0001\nCustomer: Sarah Mitchell (s.mitchell@corp.com)\n  
|   Subject: Double billing...\nBody: I am absolutely furious..."  
|  
| dialGateway.chat(ANALYSIS_SYSTEM_PROMPT, contextString)  
|   → POST https://ai-proxy.lab.epam.com/openai/deployments/  
|       gpt-5-mini-2025-08-07/chat/completions?api-version=2025-04-01-preview  
|   → raw String: "{\"category\":\"BILLING\",\"sentimentScore\":\"1,...}"  
|  
| parser.parse(rawJson, TicketAnalysisResponseDTO.class)  
|   → TicketAnalysisResponseDTO object  
|  
| WRITE-BACK (this is the key mutation):  
|   ticket.setSentimentScore(result.getSentimentScore())  
|   ticket.setSentimentLabel(result.getSentimentLabel())  
|   ticket.setCategory(result.getCategory())  
|   ticket.setPriority(result.getSuggestedPriority())  
|   ticket.setEscalationRequired(result.isEscalationRequired())  
|   if (escalationRequired && status == OPEN) → ticket.setStatus("ESCALATED")  
|   ticketRepository.save(ticket) ← persisted in H2  
|  
└─ return ResponseEntity.ok(result)
```

The write-back is the architectural differentiator. The AI call is not read-only. It mutates the entity. Future reads of this ticket from any endpoint will reflect the AI-enriched state.

4.6 Dashboard — Hybrid Architecture (Computed + AI)

The dashboard is the most architecturally interesting endpoint because it combines two data sources:

```
GET /api/support/dashboard
|
▼
SupportAiService.getDashboard()
|
├─ ticketRepository.findAll() – full table scan on H2
|
├─ Java stream aggregations (no AI involved):
|   openCount, inProgressCount, resolvedCount, escalatedCount
|   criticalCount, highCount, avgSentiment
|   categoryBreakdown (Map<String, Long> via groupingBy)
|   escalationRate, resolutionRate
|
├─ Build statsContext string (structured plain text):
|   "Support Queue Statistics:\nTotal: 20 | Open: 8 | ..."
|
├─ dialGateway.chat(DASHBOARD_SYSTEM_PROMPT, statsContext)
|   AI receives numbers, not ticket content (PII stays in DB)
|   AI returns: queueHealthScore, queueStatus, topIssues, aiRecommendations
|
└─ Merge: SupportDashboardResponseDTO.builder()
    .totalTickets(...) ← from Java aggregation
    .queueHealthScore() ← from AI
    .aiRecommendations() ← from AI
    .categoryBreakdown() ← from Java
    .build()
```

Privacy design note: For the dashboard, the AI receives only aggregate statistics, not ticket content or PII. This is a deliberate data minimisation choice — the AI only needs numbers to assess queue health. Contrast with `analyzeTicket()` where full ticket body is required for sentiment analysis.

5. RAG Architecture — Codebase Q&A

`POST /api/ai/ask` implements a lightweight Retrieval-Augmented Generation pattern over the source code.

```
User question: "How does pagination work in TaskController?"
|
▼
AiService.askQuestion()
|
├─ loadSourceFilePaths()
|   Files.walk(src/main/java) → up to 20 .java files (sorted alphabetically)
|
├─ buildContext()
|   Concatenate file contents with === filename === headers
|   Hard cap at 50,000 characters (truncation with "// [truncated]" marker)
|
├─ userMessage = "Context from the codebase:\n\n{context}\nQuestion: {question}"
|
└─ dialGateway.chat(SYSTEM_PROMPT, userMessage)
    → Free-text answer about the codebase
```

Tradeoffs vs. vector-based RAG:

Dimension	This approach (full-text)	Vector RAG
-----------	---------------------------	------------

Setup complexity	Zero — reads files from disk	High — chunking, embedding, vector DB
Precision	Lower — includes irrelevant files	Higher — similarity-ranked retrieval
Latency	Higher for large codebases	Consistent, indexed
Freshness	Always current (reads live files)	Requires re-indexing on code change
Cost	Higher token usage per query	Lower — only relevant chunks
Dependencies	None	Embedding model + vector store

For a codebase with 20–50 classes, full-text injection at 50K char cap is a pragmatic choice. The pattern would switch to vector RAG at the point where the codebase no longer fits in one prompt window.

6. Data Model

6.1 SupportTicket Entity

support_tickets		
└ id	BIGINT (PK, auto-increment)	
└ ticket_number	VARCHAR (UNIQUE, NOT NULL)	– "TKT-0001" format
└ customer_name	VARCHAR (NOT NULL)	
└ customer_email	VARCHAR (NOT NULL)	
└ subject	VARCHAR (NOT NULL)	
└ body	TEXT (NOT NULL)	– full message content
└ category	VARCHAR (NOT NULL, default="GENERAL")	
	– BILLING TECHNICAL ACCOUNT COMPLAINT REFUND GENERAL	
└ status	VARCHAR (NOT NULL, default="OPEN")	
	– OPEN IN_PROGRESS ESCALATED RESOLVED CLOSED	
└ priority	VARCHAR (NOT NULL, default="MEDIUM")	
	– LOW MEDIUM HIGH CRITICAL	
└ sentiment_score	INTEGER (nullable)	– null until AI analysis runs
└ sentiment_label	VARCHAR (nullable)	– null until AI analysis runs
└ escalation_required	BOOLEAN (default=false)	
└ source	VARCHAR (default="EMAIL")	– EMAIL CHAT PHONE WEB
└ assigned_agent	VARCHAR (nullable)	
└ created_at	TIMESTAMP (auto, immutable)	
└ updated_at	TIMESTAMP (auto, updated on every save)	

Nullable AI fields design: `sentimentScore` and `sentimentLabel` are nullable by design. A null value means "not yet analyzed" — distinguishable from a score of 5 (NEUTRAL). The "Before AI" UI page exploits this to show the raw pre-intelligence state.

Status state machine:

OPEN → IN_PROGRESS → RESOLVED → CLOSED
└→ ESCALATED → RESOLVED → CLOSED

The `ESCALATED` state is the only one triggered automatically by AI (via write-back in `analyzeTicket`). All other status transitions are manual via `PUT /api/support/tickets/{id}/status` .

7. API Surface

7.1 CRUD Endpoints (`SupportTicketController`)

Method	Path	Description
--------	------	-------------

POST	/api/support/tickets	Create ticket
GET	/api/support/tickets	List all (optional ?status= filter)
GET	/api/support/tickets/{id}	Get by ID
PUT	/api/support/tickets/{id}/status	Update status
DELETE	/api/support/tickets/{id}	Delete
GET	/api/support/tickets/escalation-queue	All escalationRequired=true tickets

7.2 AI Endpoints (SupportAiController)

Method	Path	AI call type	Mutates DB?
POST	/api/support/ai/analyze	Free-text → TicketAnalysisResponseDTO	No
POST	/api/support/tickets/{id}/analyze	Ticket context → TicketAnalysisResponseDTO	Yes
POST	/api/support/tickets/{id}/draft-reply	Ticket context → DraftReplyResponseDTO	No
GET	/api/support/dashboard	Queue stats → SupportDashboardResponseDTO	No
POST	/api/support/tickets/bulk-triage	N ticket summaries → BulkTriageResponseDTO	No

7.3 RAG Endpoint (AiController)

Method	Path	Description
POST	/api/ai/ask	Natural language question about the codebase

8. Error Handling Architecture

All exceptions funnel through a single `@RestControllerAdvice` :

Exception Type	→ HTTP Status	→ When
ResourceNotFoundException	→ 404	– ticket/task ID not found
MethodArgumentNotValidException	→ 400	– @Valid constraint failure
AiParsingException	→ 502	– AI returned non-parseable output
HttpStatusCodeException	→ 503	– DIAL returned 4xx/5xx
Exception (catch-all)	→ 500	– anything else

502 vs 503 distinction:

- `502 Bad Gateway` = DIAL responded but the model output couldn't be parsed (AI logic error)
- `503 Service Unavailable` = DIAL itself returned an HTTP error (infrastructure error)

Both signal that the failure is downstream of this service, not a client error. This matters for observability — a 502 spike means the prompt schema needs tuning; a 503 spike means DIAL is degraded or VPN is disconnected.

ErrorResponse DTO:

```
{
  "timestamp": "2026-08-02T10:15:32",
  "status": 502,
  "error": "Bad Gateway",
}
```



```
"message": "AI returned an unparseable response: Unexpected character...",
"path": "/api/support/ai/analyze"
}
```

9. Multi-Agent SDLC Architecture (Claude Code Layer)

SupportIQ was built using a multi-agent AI development pattern. The `.claude/` directory contains the entire AI development infrastructure used to build the application itself.

9.1 Specialist Agents (`.claude/agents/`)

Agent	Role
taskflow-architect	Architecture decisions, package design, dependency trade-offs
taskflow-reviewer	Pre-merge code quality review, convention audits
taskflow-tester	JUnit 5 / MockMvc test generation
support-analyst-agent	Support AI prompt engineering, sentiment/escalation logic
demo-guide-agent	Demo scripting, management talking points, ROI framing
support-triage-agent	Queue management, SLA analysis, triage strategy

Each agent is a Markdown file with a custom system prompt. When invoked, it runs with the full Claude Code toolset but with domain-specific reasoning context.

9.2 Orchestration Workflows (`.claude/workflows/`)

Workflow	Orchestration pattern	Description
feature-implementation	Sequential → parallel per layer	Research → Plan → Implement (controller/service/test in parallel) → Review
code-review	4 parallel agents	Conventions, validation, tests, API design — simultaneously
test-generation	Gap analysis → parallel writers	Identify untested paths → write tests per class in parallel
support-ai-insights	Gather → 4 parallel analyses → synthesize	Live queue data → Queue Health + Risk + Sentiment + Patterns → Executive briefing
demo-prep	Sequential verification	Verify all AI endpoints respond → generate talking points from real data

support-ai-insights workflow architecture (the most complex):

Phase 1 – Gather (1 agent, sequential)

- └ Fetches `/api/support/tickets` + `/escalation-queue` + `/dashboard`
Returns structured summary string

Phase 2 – Analyze (4 agents, parallel)

- └ analyze-queue-health → score/findings/recommendations
 - └ analyze-escalation-risk → score/findings/recommendations
 - └ analyze-sentiment-trends → score/findings/recommendations
 - └ analyze-category-patterns → score/findings/recommendations
- All receive the same gathered data. No cross-agent communication.

Phase 3 – Synthesize (1 agent, sequential, after barrier)
└─ Receives all 4 analyses, dimension scores, urgentActions
Returns executive briefing with overallHealthScore, businessRisks, wins

The parallel phase (Phase 2) uses a **barrier pattern** — all 4 analyses must complete before synthesis begins, because the synthesizer needs cross-dimensional findings to write a coherent briefing.

9.3 Automated Quality Hooks (`.claude/hooks/`)

Hook	Fires on	Action
post-java-edit	Any .java file edited	<code>mvn compile -q</code> — catch compilation errors immediately
pre-commit	Before git commit	<code>mvn test -q</code> — block commit if any test fails

The pre-commit hook means the test suite is a hard gate on every commit. 103 tests must pass before any code enters git history.

10. Testing Architecture

```
test/
├─ TaskControllerTest.java      - @WebMvcTest + MockMvc - all CRUD + validation
├─ TaskServiceTest.java        - @ExtendWith(Mockito) - unit tests, no Spring context
├─ NoteControllerTest.java     - @WebMvcTest + MockMvc
├─ NoteServiceTest.java       - Unit tests
├─ AiControllerTest.java       - @WebMvcTest, MockBean for AiService
├─ SupportTicketControllerTest.java - @WebMvcTest, all ticket endpoints + error cases
└─ SupportAiControllerTest.java - @WebMvcTest, MockBean for SupportAiService
```

Test conventions:

- `@WebMvcTest` for all controller tests — loads only the web layer, not the full context
- `@MockBean` (never `@Mock`) in `@WebMvcTest` — Spring's proxy-aware mock injection
- `@Transactional` not applied to tests — each test operates on an isolated `MockMvc` request
- `@Profile("!test")` on `SupportDataInitializer` — demo seed data does not run during tests
- No actual DIAL calls in tests — `SupportAiService` is fully mocked at the controller test level

What is NOT unit-tested (by design):

- `DialGateway` — testing the HTTP call to DIAL would require a mock server. Covered by integration.
- `AiResponseParser` — could be unit-tested easily; relies on Jackson correctness (already tested by Jackson itself).
- The system prompts — prompt quality is validated by running the demo, not by unit tests.

11. Frontend Architecture

The UI is a **single-file SPA** (`src/main/resources/static/index.html` , ~1000 lines).

Key design decisions:

- No framework, no npm, no build step — served directly by Spring Boot's static resource handler
- Relative API URL (`BASE = '/api/support'`) — no CORS issues, same origin as the server
- All state in vanilla JS module-level variables (`curPage` , `selTriage` , `curFilter`)
- Router pattern: `go(page)` toggles `.active` class on page divs and nav items, then calls the load function
- Fetch wrapper: `call(method, path, body)` → `fetch(BASE + path)` → throws on non-2xx with parsed error message
- Toast notification system for async feedback (3.5s auto-dismiss)

Pages and their API dependencies:

Page	APIs called
Dashboard	GET /dashboard
All Tickets	GET /tickets?status=, POST /{id}/analyze, POST /{id}/draft-reply
Before AI (Manual)	GET /tickets, POST /{id}/analyze
Analyze Email	POST /ai/analyze
Bulk Triage	GET /tickets?status=OPEN, POST /tickets/bulk-triage
Escalation Queue	GET /tickets/escalation-queue, POST /{id}/draft-reply, PUT /{id}/status

12. Scalability & Production Considerations

The current architecture is optimised for **correctness and demonstrability**, not throughput. These are the known production gaps:

12.1 Synchronous AI Calls

Every AI endpoint is synchronous and blocking. A single `dialGateway.chat()` call can take 1–5 seconds depending on DIAL load and response length. Under concurrent load:

- **Issue:** Servlet threads are blocked during AI response wait
- **Solution path:** Introduce `@Async` service methods + `CompletableFuture`, or move to WebFlux + reactive `WebClient`

For bulk triage with 50 tickets, the current implementation sends one large prompt. A production variant would:

1. Chunk tickets into groups of 10
2. Send parallel async calls
3. Merge and re-rank results

12.2 Database

H2 in-memory is wiped on restart. Production replacement: PostgreSQL with Flyway migrations. The JPA layer requires no code changes — only `application.properties` and `pom.xml` dependency updates. `spring.jpa.hibernate.ddl-auto=update` would change to `validate`.

12.3 AI Response Caching

Dashboard calls the AI on every `GET /dashboard`. With 20 tickets, this generates ~500 tokens per call. A production pattern:

- Cache dashboard result in Redis with a 60-second TTL
- Invalidate on any ticket status change via Spring `@CacheEvict`

12.4 API Key Security

`dial.api-key` is currently in `application.properties` (committed to repo in the demo). Production requirement:

- Inject via environment variable: `${DIAL_API_KEY}`
- Or use Spring Cloud Config Server / Vault

12.5 PII in AI Context

`analyzeTicket()` sends the full ticket body including customer name and email to DIAL. For GDPR compliance in a production deployment:

- Add a `PiiRedactor` component that replaces `customer_email`, `customer_name` patterns before sending
- Or use DIAL's PII-stripping middleware if available in the enterprise configuration

13. Architecture Decision Records

ADR-001: RestClient over WebClient

Decision: Use `spring-web` `RestClient` (blocking) rather than `WebClient` (reactive).

Reason: The service layer uses `@Transactional` and JPA — which are inherently blocking. Mixing reactive `WebClient` with blocking JPA in the same call chain creates thread-starvation risk on Reactor schedulers. Using blocking `RestClient` keeps the threading model uniform and predictable.

Consequence: AI calls block a servlet thread for the duration of the DIAL response. Acceptable for demo scale; addressed by async patterns for production.

ADR-002: No AI SDK dependency

Decision: Zero AI-specific dependencies. No `spring-ai`, no `langchain4j`, no Anthropic SDK.

Reason: Demonstrate that LLM integration is fundamentally an HTTP call with structured I/O — not a framework problem. This keeps the architecture understandable to any Java engineer and avoids SDK version lock-in.

Consequence: No built-in retry logic, streaming, or tool use. `AiResponseParser` handles what SDKs would handle automatically.

ADR-003: System prompts as static constants

Decision: `private static final String` constants in `SupportAiService`, not database rows or config files.

Reason: Prompts are code. They encode business logic (what counts as HIGH risk, what triggers escalation). They must be version-controlled, reviewed in PRs, and testable alongside the logic that uses them.

Consequence: Changing a prompt requires a code deploy. This is intentional — uncontrolled prompt changes in production are a risk.

ADR-004: Write-back on analyzeTicket

Decision: `analyzeTicket()` mutates the `SupportTicket` entity with AI output and calls `ticketRepository.save()`.

Reason: An analysis that is not persisted has no durable effect. The next agent who reads this ticket gets no benefit from the AI work done. Persistence makes the enrichment cumulative and queryable.

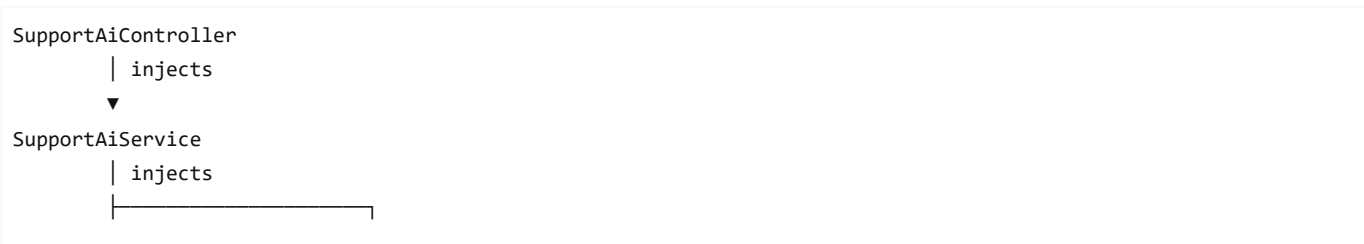
Consequence: Calling `analyzeTicket()` multiple times on the same ticket will overwrite previous AI output. This is acceptable — re-analysis is always on the current ticket state.

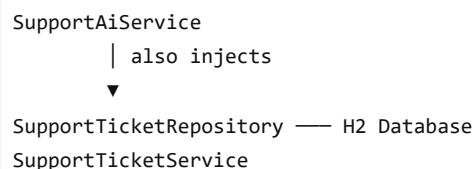
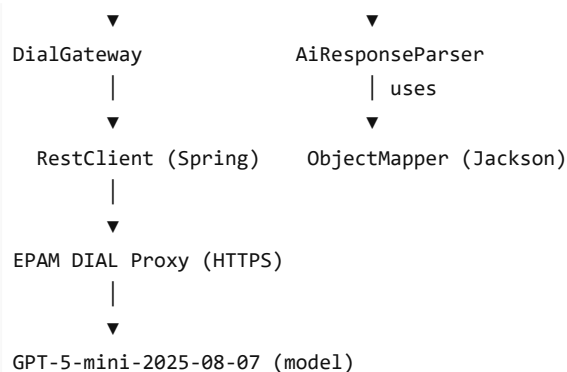
ADR-005: AiParsingException → 502 not 500

Decision: Map `AiParsingException` to HTTP 502, not 500.

Reason: 502 (Bad Gateway) accurately represents the failure mode: the upstream service (DIAL/model) returned a response that this gateway could not process. HTTP 500 would imply an internal application bug. Clients observing a 502 know to retry or check DIAL, not this application.

14. Component Dependency Graph





No service-to-service lateral calls. `SupportAiService` reaches the repository directly — same pattern used throughout this codebase. This avoids introducing service-layer coupling that would complicate transaction boundaries.

15. Quick Reference — All Endpoints

TaskFlow Core

GET	/api/tasks	– paginated list (status, priority filters)
POST	/api/tasks	– create task
GET	/api/tasks/{id}	– get task
PUT	/api/tasks/{id}	– update task
DELETE	/api/tasks/{id}	– delete task
GET	/api/tasks/{id}/notes	– list notes
POST	/api/tasks/{id}/notes	– add note
DELETE	/api/tasks/{id}/notes/{nid}	– delete note

RAG

POST	/api/ai/ask	– codebase Q&A via RAG
------	-------------	------------------------

SupportIQ CRUD

POST	/api/support/tickets	
GET	/api/support/tickets	(?status= filter)
GET	/api/support/tickets/{id}	
PUT	/api/support/tickets/{id}/status	
DELETE	/api/support/tickets/{id}	
GET	/api/support/tickets/escalation-queue	

SupportIQ AI

POST	/api/support/ai/analyze	– raw email → intelligence report
POST	/api/support/tickets/{id}/analyze	– stored ticket → enrich + write-back
POST	/api/support/tickets/{id}/draft-reply	– stored ticket → drafted reply
GET	/api/support/dashboard	– queue stats + AI health assessment
POST	/api/support/tickets/bulk-triage	– N tickets → urgency ranking

Tooling

GET	/swagger-ui.html	– interactive API documentation
-----	------------------	---------------------------------

GET	/h2-console	– in-memory database browser
GET	/api-docs	– OpenAPI JSON spec

Last updated: Day 8, AI-Native Engineering Series — SupportIQ platform complete